

Resolving GitLab CI/CD Disk-Quota Errors for CUDA Libraries

Optimizing Astral uv and micromamba configurations within restricted shared runners for automated machine learning verification pipelines

datalab@unimib.it

Project:	dve-sample-py
Document:	Python-Q2b-GitLab-CI-CUDA-Setup.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py/notes/howtos/
Date:	July 16, 2026

Keywords: GEN, GitLab-CI, CUDA, PyTorch, Astral-uv, Micromamba

Abstract

This technical report addresses the disk-quota constraints encountered in GitLab CI/CD pipelines when deploying complex high-performance computing environments. Specifically, it investigates storage exhaustion caused by massive CUDA-bundled binary wheels during verification workflows on shared runners restricted to a 20 GB storage limit.

To resolve this failure without compromising the simplicity of local environment initialization via `uv sync`, a dual optimization strategy is proposed. The framework details the refactoring of `pyproject.toml` and `.gitlab-ci.yml` to decouple local GPU-enabled configurations from cloud-based validation steps.

Furthermore, an alternative deployment architecture utilizing `micromamba` is evaluated. This analysis weighs the trade-offs regarding externalizing CUDA dependencies, dependency isolation, reproducibility, and structural overhead in derived templates.

Contents

TOC	5
Q:1	6
Q:1 - CI/CD Disk-Quota Resolution for CUDA libraries	6
Role	6
Technical Constraints	6
Problem Statement	7
Objective	7
Source Files	7
Deliverables	8
1. Refactored pyproject.toml	8
2. Refactored .gitlab-ci.yml	8
3. conda-env.yaml for micromamba Alternative	8
Output Format	8
A:1 (Claude)	10
Root-Cause Analysis	10
Deliverable 1 — Refactored pyproject.toml	10
Index Override Mechanism	10
NVIDIA Compatibility Notes	11
Refactored pyproject.toml	11
Deliverable 2 — Refactored .gitlab-ci.yml	18
Strategy Summary	18
Cache Optimisation Detail	19
Refactored .gitlab-ci.yml	19
Deliverable 3 — conda-env.yaml for the micromamba Alternative	22
Mechanism	22
Pros	22
Cons	22
Impact on Derived Projects	23
Refactored conda-cuda-env.yaml	23
Additional Notes	25
uv.lock and Cache Key Stability	25
sentence-transformers and Secondary torch Pulls	25
python:3.14-slim Base Image	25
uv Self-Update in CI	25
CUDA 13.1 and PyTorch Version Availability	26
References	26

A:1 (Gemini)	27
1. Refactored Project Configuration	27
2. Refactored Pipeline Definition	28
3. The Micromamba Alternative Analysis	29
Mechanism /ag	30
Pros /ag	30
Cons /ag	30
Impact on Derived Projects /ag	31
Additional Notes on Production Deployment	31
A:1 (ChatGPT)	32
Refactoring Strategy	32
Missing Attachment	32
1. Refactored pyproject.toml	33
Design Goals	33
Key Architectural Change	33
Refactored pyproject.toml	34
Environment Behaviour After Refactor	37
dev	37
test	37
prod	37
int (GitLab CI)	38
NVIDIA Compatibility Considerations	38
uv-Only GPU Model	38
Compatibility Matrix	38
Important Constraint	39
2. Refactored .gitlab-ci.yml	39
CI Design Principles	39
Recommended CI Strategy	40
Refactored .gitlab-ci.yml	40
Why This Solves the Problem	41
Before	41
After	41
Additional CI Optimisations	42
Use --no-extra	42
Use UV_LINK_MODE=copy	42
Disable bytecode compilation	42
Avoid Jupyter in CI	42
3. conda-env.yaml for micromamba Alternative	42
Mechanism	43
Representative conda-env.yaml	43
Required PyTorch Change for micromamba	44
Pros of micromamba	44
Massive Disk Reduction	44
Better Container Layer Reuse	45

Cleaner HPC Integration	45
More Reproducible GPU ABI	45
Cons of micromamba	45
More Complex Developer Setup	45
Shell Initialisation Burden	46
CI Complexity	46
Solver Overhead	46
Impact on Derived Projects	46
uv-Only Model	46
micromamba Model	47
Recommended Final Architecture	47
Recommended Hybrid Strategy	47
Additional Notes	48
Recommended Future Improvement	48
Strong Recommendation for Containers	48
Avoid triton in Non-Training Environments	48
References	48
A:1 (DeepSeek)	50
Missing Attachments	50
1. Refactored pyproject.toml /ad	50
How the virtual environment is set up for GPU-enabled environments	51
2. Refactored .gitlab-ci.yml /ad	52
Why this solves the disk-quota problem	53
3. conda-env.yaml for micromamba Alternative /ad	54
Mechanism: how micromamba decouples CUDA from uv sync	54
Pros /ad	54
Cons /ad	55
Impact on derived projects	55
Additional Notes /ad	55
References /ad	56
A:1 (Perplexity)	57
Refactored pyproject.toml /ap	57
Why this fixes the template problem	63
Refactored .gitlab-ci.yml /ap	63
What changed	64
Runner footprint tuning	64
Micromamba alternative	64
Mechanism /ap	65
Pros /ap	65
Cons /ap	65
Impact on derived projects /ap	65
conda-env.yaml for micromamba	65
Compatibility notes	66

NVIDIA versioning	66
Additional Notes /ap	66
A:1 (Vibe)	68
1. Root Cause Analysis	68
Current Setup	68
2. Refactored Files	68
2.1. Refactored pyproject.toml	68
2.2. Refactored .gitlab-ci.yml	70
3. micromamba Alternative Analysis	72
3.1. Mechanism	72
3.2. Pros	72
3.3. Cons	73
3.4. Impact on Derived Projects	73
3.5. conda-env.yaml for micromamba	73
4. NVIDIA Compatibility Matrix	74
5. Additional Notes /av	74
5.1. Why Not CPU-Only PyTorch Everywhere?	74
5.2. Why Not Cache .venv in CI?	74
5.3. uv vs. micromamba for CUDA	74
5.4. Recommended Approach	75
6. Footnotes	75
Q:2	76
Q:2 - TODO:(q2-title)	76
Question Prompt 2	76
A:2 (Claude)	77
A:2 (Gemini)	78
A:2 (ChatGPT)	79
A:2 (DeepSeek)	80
A:2 (Perplexity)	81
A:2 (Vibe)	82
A:a	83
A:a - TODO:(aa-title)	83
Appendix a	83

TOC

1. Q:1 - CI/CD Disk-Quota Resolution for CUDA libraries
 - see: CI/CD Disk-Quota Resolution: Refactoring Analysis (Claude)
 - see: CI/CD Disk Quota Resolution for ML (Gemini)
 - see: CI GPU Dependency Refactor (ChatGPT)
 - see: Refactored GPU ML Project Configurations (DeepSeek)
 - see: Refactored GPU ML Project Configurations (Perplexity)
 - see: Refactored CI/CD Pipeline and micromamba Analysis (Vibe)
2. Q:2 - TODO:(q2-ref)
 - see: TODO:(a2-ref-claude) (Claude)
 - see: TODO:(a2-ref-gemini) (Gemini)
 - see: TODO:(a2-ref-chatgpt) (ChatGPT)
 - see: TODO:(a2-ref-deepseek) (DeepSeek)
 - see: TODO:(a2-ref-perplexity) (Perplexity)
 - see: TODO:(a2-ref-vibe) (Vibe)
3. A:a - TODO:(appendix-a)

Q:1

Q:1 - CI/CD Disk-Quota Resolution for CUDA libraries

^

Role

You are a Senior DevOps Engineer and High-Performance Computing (HPC) specialist. Your expertise covers containerised environments for Machine Learning, specifically optimising NVIDIA CUDA stacks for PyTorch using micromamba and Astral uv. You are also an expert in GitLab CI/CD pipeline configuration.

Technical Constraints

- *Project Environments:*
 1. dev: interactive local development (native, un-containerised, Ubuntu Linux, GPU available)
 2. test: batch local development (native, un-containerised, Ubuntu Linux, GPU available)
 3. prod: batch runtime on a multi-node cluster (containerised via Podman, Ubuntu Linux, GPU available)
 4. int: GitLab SaaS (GitLab.com) CI/CD verification gate before MR merging to mainline (batch, shared cloud runner, disk quota: 20 GB)
- *Development Environment:* Rootless Podman, Ubuntu-based images.
- *CI/CD Environment:* GitLab SaaS pipeline defined in `.gitlab-ci.yml`, shared "Ultimate" account.
- *MR Verification Pipeline:* runs on the develop integration branch and performs:
 1. Static type checking via `pyright`
 2. Lint checks via `ruff check`
 3. Format checks via `ruff format`
 4. Unit tests (no GPU required) via `pytest`
- *The "Simple-as-possible" Rule:* This project is a template for multiple research projects sharing the same prerequisites. Derived projects must be able to set up a GPU-enabled Python virtual environment with a single command:

```
uv sync --all-extras --all-groups
```

No complex, project-specific post-sync steps are permitted.

- *Privileges*: In dev and test environments, only user-level access is granted (\$HOME-relative paths only). Inside Podman containers, root (uid=0) access is available.
- *Package Management*:
 - uv manages Python version selection and virtual environments.
 - micromamba is a candidate alternative for installing cuda-toolkit, cudnn, and libblas, externalising CUDA from the uv-managed virtual environment.
 - *Crucial*: if micromamba is used, PyTorch must be installed by uv against the system-provided CUDA libraries, *not* via CUDA-bundled wheel downloads.

Problem Statement

The CI/CD pipeline fails in early stages because the `.venv` restored from cache exceeds the 20 GB disk quota of the GitLab SaaS shared runner. The root cause is that torch, when installed via `uv sync`, pulls in massive CUDA-bundled binary wheels (including triton shared libraries).

A hard switch to a CPU-only torch variant is explicitly prohibited by the *"Simple-as-possible"* Rule: derived GPU-enabled projects must continue to use `uv sync --all-extras --all-groups` without modification.

The observed pipeline error is:

```
WARNING: .venv/lib/python3.14/site-packages/triton/_C/libtriton.so:
write: no space left on device
```

```
ERROR: Job failed (system failure): Error response from daemon:
symlink [...]: no space left on device (docker.go:898:0s)
```

Objective

Provide a concrete refactoring of `pyproject.toml` and `.gitlab-ci.yml` that resolves the disk-quota failure while respecting the *"Simple-as-possible"* Rule.

Additionally, analyse the micromamba alternative setup as a means of externalising the CUDA dependency from uv virtual environment management. Structure the analysis as follows:

- *Mechanism*: how micromamba decouples CUDA from `uv sync`.
- *Pros*: benefits for disk size, portability, and reproducibility.
- *Cons*: additional setup burden (e.g. `~/.bashrc` / `~/.zshrc` modifications, `conda activate` in CI steps, solver overhead).
- *Impact on derived projects*: what a template consumer must do differently.

Source Files

The following configuration files are provided as attachments. If any attachment is absent, state which file is missing and proceed by generating a representative example based on the constraints

above.

1. *Original pyproject.toml* — current project configuration.
2. *Original .gitlab-ci.yml* — current CI/CD pipeline definition.
3. *Original conda-env.yaml* — existing micromamba environment specification.

Deliverables

1. Refactored pyproject.toml

- Produce a PEP 508 / PEP 621 compliant, uv-compatible project file.
- Describe how the virtual environment is set up for GPU-enabled environments, both native and containerised.
- Discuss NVIDIA compatibility considerations: GPU family, driver version, cuDNN, and PyTorch versioning — for both the uv-only and micromamba-assisted cases.

2. Refactored .gitlab-ci.yml

- Specify the correct `uv sync` command and flags for the CI context.
- Optimise disk usage and job duration given the 20 GB shared runner quota, focusing on the verification nature of the pipeline (no GPU, no large binary wheels needed).
- Tune cache configuration and gitlab-runner settings to minimise footprint.

3. conda-env.yaml for micromamba Alternative

- Define a micromamba/conda environment containing `cuda-toolkit=13.1`, `cuda`, and `libblas`.
- Ensure compatibility with NVIDIA Driver 525+ and the CUDA 12.x Toolkit (CTK).
- Note any version pinning required for PyTorch compatibility.

Output Format

- Reply in clear, formatted GitLab Flavored Markdown (GLFM) with lint-valid structure:
 - Place all codeblock delimiters (```) at line start; avoid nested codeblocks.
 - Use *underscore markup* for emphasis.
 - Prefer nested headings over bold/asterisk text markup.
 - Use only dashes for unordered lists, with correct indentation.
 - Insert a blank line after every heading, list block, and codeblock.
- Ignore document-pipeline formatting markup:
 - `<details><summary>` HTML blocks
 - `{=latex}` codeblocks
 - `[!tip]` / `[!note]` block quotes
 - Code-folding tags (`{{{ / }}})`
 - Internal cross-reference links (e.g. `[^]`)

- At the end, provide Markdown footnotes for all referenced online documentation, linked inline where appropriate. Prefix all footnote identifiers with `rf-` to avoid clashes with document-level references.
- Add any important information not explicitly requested above in an *"Additional Notes"* section.

A:1 (Claude)

^ *CI/CD Disk-Quota Resolution: Refactoring Analysis*

Root-Cause Analysis

The failure originates from a combination of three factors:

- torch (CUDA variant) from <https://download.pytorch.org/whl/cu130> lists triton as a hard runtime dependency. triton ships a ~1.5 GB native shared library (libtriton.so).
- The current before_script runs `uv sync --dev`, which installs the full project dependency closure including every main-project dependency — torch, torchvision, and their transitive closure — even though no CI job exercises GPU code paths.
- The `.venv` directory is included in the GitLab cache, meaning the virtual environment is serialised and deserialised across jobs, doubling the effective disk pressure beyond the 20 GB quota during the cache-restore phase.

The targeted fix is to redirect the named torch index to the CPU wheel server inside CI via a uv-native environment variable override, so no CUDA-bundled binary is downloaded at all. No source-level change to the `uv sync` invocation consumed by downstream projects is required.

Deliverable 1 — Refactored `pyproject.toml`

Index Override Mechanism

uv exposes every named index defined under `[[tool.uv.index]]` as an overridable environment variable¹. For an index named `torch-gpu`, the corresponding variable is:

```
UV_INDEX_TORCH_GPU=https://download.pytorch.org/whl/cpu
```

The name is uppercased and all hyphens are replaced by underscores. Setting this variable in the GitLab CI job causes uv to resolve torch and torchvision against the CPU wheel server while every downstream consumer that omits the variable continues to receive CUDA wheels unchanged. The `uv sync --all-extras --all-groups` contract is preserved verbatim.

¹Astral uv — *Index configuration and environment variable overrides*: <https://docs.astral.sh/uv/configuration/indexes/>

NVIDIA Compatibility Notes

uv-only Case (CUDA-bundled Wheels)

- The `cu130` wheel index targets CUDA Toolkit 13.0 (driver ≥ 580 on Linux²).
- PyTorch 2.10.x requires cuDNN 9.x, NCCL 2.21+, and the matching `libcuda.so` ABI from the driver package.
- No host-level CUDA installation is required: all CUDA runtime libraries are bundled inside the wheel tree (in `.venv/lib/python3.x/site-packages/nvidia/`).
- `triton` (GPU kernel compiler) is a hard dependency of every CUDA-enabled torch build and *must* be present for JIT-compiled GPU kernels at runtime, but is not needed during CI verification.

micromamba-assisted Case

- CUDA is provided at the OS level by the conda environment; `uv` must therefore resolve torch against a wheel that links dynamically against the system `libcuda.so` / `libcudart.so`.
- The `download.pytorch.org/whl/cu130` index ships exactly those slim wheels: they are compiled against CUDA 13.0 headers but *do not rebundle* the CUDA runtime.
- `LD_LIBRARY_PATH` must include the conda environment's `lib/` directory before `uv sync` is invoked, so the linker resolution succeeds at install time.
- `triton` is still installed as a transitive dependency; its disk footprint is unavoidable in GPU environments, but is contained within the `uv`-managed `.venv`.

Refactored `pyproject.toml`

The only structural change to the original file is renaming the index from `torch130` to `torch-gpu` to produce a readable, intent-revealing environment variable name. All other content is preserved.

```
# =====
# Build system (PEP 518 / PEP 517)
# =====
[build-system]
requires = ["hatchling>=1.21"]
build-backend = "hatchling.build"

# =====
# Project metadata (PEP 621)
# =====
[project]
name = "dve_sample_py"
version = "0.1.0"
description = "TODO(title): TITLE OF THE RESEARCH WORK / TITOLO DEL LAVORO DI RICERCA"
readme = "README.md"
requires-python = ">=3.10,<3.15"
```

²NVIDIA CUDA Compatibility — Driver and toolkit version matrix: <https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/>

```
authors = [{ name = "Data Science Lab DEMS/datalab", email = "datalab@unimib.it" }]

# =====
# Dependencies (PEP 508 strings)
# =====

dependencies = [

    # --- ML frameworks ---
    # Index source: controlled by [[tool.uv.index]] name = "torch-gpu".
    # In GPU environments (dev / test / prod) the index resolves to:
    #   https://download.pytorch.org/whl/cu130
    # In CI set:
    #   UV_INDEX_TORCH_GPU=https://download.pytorch.org/whl/cpu
    # to redirect to the CPU wheel server, eliminating triton and all
    # CUDA-bundled libraries from the virtual environment.
    "torch==2.10.*",
    "torchvision==0.25.*",

    # --- data ---
    "numpy",
    "pandas",
    "pandas-datareader",
    "openpyxl",
    "pyarrow",
    "polars",

    # --- sql ---
    "SQLAlchemy",
    "psycopg2-binary",
    "mysql-connector-python",

    # --- stats ---
    "scipy",
    "sympy",

    # --- plots ---
    "matplotlib",
    "plotly",
    "seaborn",
    "Pillow",

    # --- graphs ---
    "graphframes",
    "graphviz",
    "networkx",
```

```

    "igraph",
    "pyvis",
    "pydot",

    # --- system ---
    "requests",
    "oauthlib",
    "requests-oauthlib",
    "urllib3",
    "pyzmq",

    "pyyaml",
    "types-pyyaml",
    "toml",
    "types-toml",
    "piny",
    "click",
    "pathspec",

    "icecream",
    "tqdm",

    "pip",
    "more-itertools",

    # --- git dependency ---
    "rootpath @ git+https://github.com/hute37/python-rootpath@stable",
]

# =====
# Optional dependency groups (PEP 621 Extras)
# =====
[project.optional-dependencies]

gpumon = [
    "nvidia-ml-py; sys_platform == 'linux'",
]

# =====
# Project URLs
# =====
[project.urls]
documentation = "https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py/-/wikis/home"
homepage = "https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py"

```

```
repository = "https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py.git"

# =====
# Script commands
# =====

[project.scripts]
main = "dve.cli:main"
demo = "vce.cli:main"
hello = "dve.scripts.dummy.dummy_hello:main"

# =====
# Development dependency groups
# =====

[dependency-groups]

agents = [
    "aiohttp",
    "anthropic",
    "graphifyy",
    "hypothesis",
    "line-profiler",
    "memory-profiler",
    "openai",
    "pydantic",
    "scikit-learn",
    "sentence-transformers",
]

jupyter = [
    "jupyter",
    "jupyter-core",
    "jupyter-console",
    "jupyter-lsp",
    "notebook",
    "jupyterlab",
    "jupyter-text",
    "nodeenv",
    "nodejs",
    "jupyterlab-lsp",
    "jupyter-ruff",
    "lux-api",
    "ipykernel",
    "ipywidgets",
    "ipython",
    "ipython-bg",
]
```

```

    "marimo",
]

dev = [
    # --- test ---
    "pytest",
    "pytest-cov",
    "behave",
    "mock",
    "nose",
    # --- edit ---
    "pyright",
    "basedpyright",
    "tree-sitter",
    # --- lint ---
    "ruff",
    # --- misc ---
    "colorama",
    "py",
    "mypy",
    # --- packaging ---
    "setuptools",
    "wheel",
    "bumpversion",
    "twine",
]

# =====
# uv configuration
# =====

[tool.uv]
package = true

[tool.uv.sources]
# Both packages resolved from the "torch-gpu" named index.
# Override in CI with: UV_INDEX_TORCH_GPU=https://download.pytorch.org/whl/cpu
torch = { index = "torch-gpu" }
torchvision = { index = "torch-gpu" }

[[tool.uv.index]]
# GPU default: CUDA 13.0-compiled wheels (driver ≥ 580).
# Rename from "torch130" to "torch-gpu" so the uv env-var override reads:
#   UV_INDEX_TORCH_GPU=<url>
name = "torch-gpu"
url = "https://download.pytorch.org/whl/cu130"

```



```
# =====
# Hatch build configuration
# =====

[tool.hatch.metadata]
allow-direct-references = true

[tool.hatch.build.targets.wheel]
packages = ["src/dve", "src/vce"]

# =====
# pytest
# =====

[tool.pytest.ini_options]
pythonpath = ["src", "tests/pytest"]
testpaths = ["tests/pytest"]
python_files = ["test_*.py"]
python_classes = ["Test*"]
python_functions = ["test_*"]
addopts = "-v --tb=short"

# =====
# mypy
# =====

[tool.mypy]
python_version = "3.10"
strict = true
ignore_missing_imports = true

# =====
# pyright
# =====

[tool.pyright]
include = ["src", "tests/pytest", "notebooks"]
exclude = [
    "notebooks/@databricks/*",
    "doc/*",
    "home/*",
    "notes/*",
    "renv/*",
    ".venv/*",
    ".agents/*",
    ".eggs/*.py",
    "**/node_modules",
    "**/__pycache__",
]
```

```
"docker",
"notes",
"data",
"logs",
"temp",
]
ignore = ["build"]
defineConstant = { DEBUG = true }
typeCheckingMode = "basic"
reportMissingImports = true

# =====
# ruff
# =====

[tool.ruff]
line-length = 100
target-version = "py310"
fix = true

[tool.ruff.lint]
unfixable = []
typing-modules = ["pandas._typing"]
exclude = [
    "notebooks/@databricks/*",
    "doc/*",
    ".eggs/*.py",
    "home/*",
    "notes/*",
    "renv/*",
    ".venv/*",
    ".agents/*",
]
select = [
    "F", "E", "W", "YTT", "B", "Q", "T10", "INT",
    "PL", "PT", "PIE", "PYI", "TID", "ISC", "TCH",
    "C4", "PGH", "RUF", "S102", "NPY002", "PERF",
    "FLY", "G", "FA", "ICN001", "SLOT", "RSE",
]
ignore = []

[tool.ruff.lint.pydocstyle]
convention = "google"

[tool.ruff.lint.isort]
combine-as-imports = true
```

```

[tool.ruff.format]
docstring-code-format = true
exclude = [
    "notebooks/@databricks/*",
    "doc/*",
    ".eggs/*.py",
    "home/*",
    "notes/*",
    "renv/*",
    ".venv/*",
    ".agents/*",
]

# =====
# Project Custom Configuration
# =====
[tool.local]
has_config = true
config = "config.yaml"

[tool.local.paths]
log_dir = "logs"

[tool.local.log]
level = "INFO"

[tool.local.hello]
salutation = "Hi"

```

Deliverable 2 — Refactored `.gitlab-ci.yml`

Strategy Summary

The refactoring applies four independent optimisations:

- *Index redirection*: `UV_INDEX_TORCH_GPU` is set to the CPU wheel server for all CI jobs. `torch` and `torchvision` resolve to small CPU builds (~200 MB combined); `triton` is not pulled as a dependency because the CPU build has no JIT compilation backend.
- *Cache scope reduction*: `.venv` is removed from the cache block. Only the `uv` download cache (`$UV_CACHE_DIR`) is persisted. Wheel archives in the download cache are order-of-magnitude smaller than extracted virtual environments; `.venv` is reconstructed cheaply from cached wheels on every job run.
- *Per-job sync scoping*: the linting job uses `--only-group dev --no-install-project` to skip

project package installation entirely — `ruff` operates on source text and has no import-time dependency on `torch`. The typing and testing jobs use `--group dev`, which installs main dependencies (including CPU `torch`) plus the dev group.

- *before_script elimination*: each job declares its own minimal `before_script` rather than sharing a maximal one, preventing heavier sync operations from running for jobs that do not need them.

Cache Optimisation Detail

The GitLab runner's 20 GB quota applies to the *working directory* at the point of cache serialisation³. Removing `.venv` from the cache has two effects:

- The cache archive shrinks from multi-gigabyte size (CUDA wheels + triton) to a few hundred megabytes (wheel tarballs in `$UV_CACHE_DIR`).
- Each job always starts from a clean virtual environment, eliminating stale-dependency bugs that arise when a cached `.venv` outlives a `pyproject.toml` change.

The uv wheel cache deduplicates content by hash⁴; reconstructing `.venv` from a warm `$UV_CACHE_DIR` typically completes in under thirty seconds.

Refactored `.gitlab-ci.yml`

```

---
# -----
# Disk-budget notes
# -----
# GitLab SaaS shared runners carry a 20 GB working-directory quota.
# The CUDA-bundled torch wheel + triton alone exceed this budget.
#
# Fix: redirect the "torch-gpu" named index to the CPU wheel server for all
# CI jobs via UV_INDEX_TORCH_GPU. Downstream consumers running on GPU hosts
# omit this variable and continue to receive CUDA wheels from the cu130 index.
#
# Cache strategy: persist only $UV_CACHE_DIR (downloaded wheel archives).
# Do NOT cache .venv — extracting from the wheel cache is fast (~30 s) and
# avoids the cache-serialisation phase that itself hits the disk quota.
# -----

```

```
image: python:3.14-slim
```

```
variables:
```

```

  # --- uv installation ---
  UV_VERSION: "latest"

```

³GitLab CI/CD — *Caching in GitLab CI/CD*: <https://docs.gitlab.com/ee/ci/caching/>

⁴Astral uv — *Caching*: <https://docs.astral.sh/uv/concepts/cache/>

```

# --- uv cache location (project-relative, included in GitLab cache) ---
UV_CACHE_DIR: "$CI_PROJECT_DIR/.cache/uv"

# --- index override: redirect "torch-gpu" to CPU wheel server in CI ---
# uv resolves UV_INDEX_<NAME_UPPER> where hyphens become underscores.
# CPU wheels do not include triton, saving ~1.5 GB per job.
UV_INDEX_TORCH_GPU: "https://download.pytorch.org/whl/cpu"

# --- disable progress bars (cleaner CI logs) ---
UV_NO_PROGRESS: "1"

# --- never use the system Python; always use uv-managed interpreter ---
UV_PYTHON_PREFERENCE: "only-managed"

# Only the uv download cache is persisted. .venv is intentionally excluded.
cache:
  key:
    files:
      - pyproject.toml
      - uv.lock
  paths:
    - .cache/uv
  policy: pull-push

stages:
  - lint
  - test

# -----
# Shared setup fragment – referenced via YAML anchors
# -----
.install_uv: &install_uv
  - curl -LsSf https://astral.sh/uv/install.sh | sh
  - export PATH="$HOME/.local/bin:$PATH"
  - uv --version

# -----
# lint stage
# -----

linting:
  stage: lint
  before_script:
    - *install_uv
    # Lint jobs operate on source text only.

```

```

    # --only-group dev : install ruff and its peers; skip main dependencies.
    # --no-install-project : do not install the project package itself.
    # Result: no torch, no torchvision, no triton, minimal disk use (~150 MB).
    - uv sync --only-group dev --no-install-project
    - df -h
script:
    - uv run ruff check .
    - uv run ruff format --check .
rules:
    - if: '$CI_COMMIT_BRANCH = "develop"'

typing:
  stage: lint
  before_script:
    - *install_uv
    # pyright requires all project imports to be resolvable, so main deps
    # (including CPU torch) must be present. UV_INDEX_TORCH_GPU redirects
    # the cu130 index to the CPU wheel server; triton is not a dep of CPU torch.
    - uv sync --group dev
    - df -h
  script:
    - uv run pyright
  rules:
    - if: '$CI_COMMIT_BRANCH = "develop"'

# -----
# test stage
# -----

testing:
  stage: test
  before_script:
    - *install_uv
    # Same sync scope as typing: main deps + dev group, CPU torch.
    - uv sync --group dev
    - df -h
  script:
    - uv run pytest -v
  rules:
    - if: '$CI_COMMIT_BRANCH = "develop"'

```

Deliverable 3 — `conda-env.yaml` for the micromamba Alternative

Mechanism

micromamba⁵ installs CUDA Toolkit, cuDNN, and BLAS as ordinary shared libraries into a conda environment prefix (e.g. `/opt/mamba/envs/cuda-base/lib`). When `LD_LIBRARY_PATH` includes that prefix *before* `uv sync` is invoked, the linker resolution during wheel installation succeeds against the system-provided libraries instead of the bundled ones.

The critical consequence for torch: wheels fetched from `download.pytorch.org/whl/cu130` are compiled against CUDA 13.0 headers but *do not re-bundle* `libcudart.so`, `libcudart.so`, or `libcudnn.so` — those are expected to be present on `LD_LIBRARY_PATH` at runtime. This is distinct from the PyPI default wheel, which re-bundles every CUDA runtime library inside `site-packages/nvidia/`.

triton remains a transitive dependency of CUDA-enabled torch builds regardless of whether CUDA is managed by conda or bundled in the wheel. Its disk footprint (~1.5 GB) is therefore *not* eliminated by the micromamba approach — it is merely relocated from the conda prefix to `.venv`.

Pros

- *System CUDA deduplication*: CUDA runtime libraries (cudart, cublas, cufft, cuDNN) live once in the conda prefix rather than being replicated inside every Python virtual environment that depends on torch. On a multi-project research cluster this can save tens of gigabytes per node.
- *Driver/toolkit version governance*: the conda environment provides a single, pinned CUDA toolkit version for all Python projects on the host. Updating CUDA becomes a conda operation rather than a per-project wheel reinstall.
- *BLAS portability*: `libblas=*=*openblas` in the conda spec ensures consistent BLAS ABI regardless of whether Intel MKL is installed on the host, which matters for reproducibility across AMD and Intel CPU families.
- *Container image size*: a Podman image that starts from a micromamba-installed CUDA base and installs only slim torch wheels is measurably smaller than one that unpacks CUDA-bundled wheels into a `venv`.

Cons

- *Shell initialisation burden*: micromamba `shell init` writes hooks into `~/.bashrc` / `~/.zshrc`⁶. Every derived project's README must document this step and every CI pipeline that uses micromamba must source the hook or call `eval "$(micromamba shell hook --shell bash)"` before activating the environment.
- *conda activate in CI*: GitLab CI shells do not source `~/.bashrc` by default. Each job must explicitly activate the conda environment or prepend the env's `bin/` to `$PATH` before invoking `uv`.
- *Solver overhead*: the conda solver (even micromamba's faster libmamba backend) adds 30–120 seconds to environment creation compared with a pure `uv resolve`. Cached conda environments

⁵micromamba — Fast, standalone conda-compatible package manager: https://mamba.readthedocs.io/en/latest/user_guide/micromamba.html

⁶micromamba — Shell initialisation: https://mamba.readthedocs.io/en/latest/user_guide/micromamba.html#shell-init

mitigate this but introduce their own cache-invalidation complexity.

- *Two-tool mental model*: project contributors must understand both micromamba (for CUDA) and uv (for Python packages). The abstraction boundary between "system libraries" and "Python packages" is not obvious to researchers unfamiliar with HPC toolchains.
- *LD_LIBRARY_PATH fragility*: if the conda prefix is not on LD_LIBRARY_PATH at uv sync time and again at runtime, torch will fail to load libcuda.so with a cryptic OSError. This is a silent correctness hazard that is absent from the bundled-wheel approach.

Impact on Derived Projects

A template consumer adopting the micromamba path must:

- Install micromamba and run `micromamba env create -f conda-cuda-env.yaml` once per host.
- Add the following to their shell profile (or to the project's `.envrc` via `direnv`):

```
export CUDA_HOME=/opt/mamba/envs/cuda-base
export LD_LIBRARY_PATH="${CUDA_HOME}/lib:${LD_LIBRARY_PATH}"
eval "$(micromamba shell hook --shell bash)"
micromamba activate cuda-base
```

- Ensure `uv sync --all-extras --all-groups` is run *after* the above exports so that uv links torch against the system CUDA rather than triggering a bundled-wheel download. The `uv sync` command itself remains unchanged.
- Any CI pipeline for the derived project that exercises GPU code must provision the micromamba environment before the `uv sync` step. Shared GitLab SaaS runners do not support this and remain limited to CPU verification (i.e. the index-override approach from Deliverable 2 still applies to CI regardless of which approach is used in dev/prod).

Refactored `conda-cuda-env.yaml`

The existing file targets CUDA 13.1 and driver ≥ 580 . The adjustments below correct two specification issues present in the original: `cutensor` lacks a channel qualifier on the `nvidia` channel, and the virtual package pin (`__cuda`) must be expressed as a conda *virtual package* constraint, not a dependency⁷.

```
---
# conda-cuda-env.yaml
# System-level CUDA / cuDNN / BLAS environment managed by micromamba.
# Targets NVIDIA driver  $\geq 580$  (required for CUDA Toolkit 13.1).
#
# Install:
# micromamba env create -p /opt/mamba/envs/cuda-base -f conda-cuda-env.yaml
#
# Activate in shell:
# export CUDA_HOME=/opt/mamba/envs/cuda-base
# export LD_LIBRARY_PATH="${CUDA_HOME}/lib:${LD_LIBRARY_PATH}"
# micromamba activate /opt/mamba/envs/cuda-base
```

⁷conda — Virtual packages: <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-virtual.html>


```

#
# PyTorch compatibility:
#   Install torch via uv from the cu130 wheel index AFTER activating this env.
#   The wheels link against libcuda.so / libcudart.so from this prefix.
#   torch ≥ 2.6 is required for CUDA 13.x support; pin accordingly in
#   pyproject.toml.

name: cuda-base

channels:
  - nvidia          # Primary source for CTK 13.x packages
  - conda-forge     # Fallback and supplementary builds

# Virtual-package constraint: the conda solver enforces that the host driver
# satisfies __cuda ≥ 13.1. This is a solver hint, not an installable package.
# Express it via the `virtual_package_spec` key (micromamba ≥ 1.5):
virtual_package_spec:
  __cuda: "≥13.1"

dependencies:

# — CUDA Toolkit —————
# Meta-package: pulls nvcc, runtime libs, and development headers as a
# matched set. Pinned to patch-level for reproducibility.
- cuda-toolkit=13.1.*

# — cuDNN —————
# cuDNN 9.x is the stable series compatible with CUDA 13.1.
# Allow minor-version float to receive security patches.
- cudnn ≥ 9, < 10

# — BLAS / Linear Algebra (CPU-side) —————
# OpenBLAS is preferred over MKL for AMD EPYC (Rome/Milan) CPUs.
# The *=*openblas build-string selector pins the shim backend consistently.
- libblas=*=*openblas
- libcblas=*=*openblas
- liblapack=*=*openblas
- libopenblas

# — GPU BLAS (cuBLAS) —————
# Provides GPU-accelerated dense linear algebra; used directly by PyTorch.
# Does not replace libblas – the two stacks are complementary.
- libcublas
- libcublas-dev

```

```

# — Compiler Toolchain —————
# nvcc on CUDA 13.x requires GCC ≤ 13 on Linux.
# gxx_linux-64=12 is a known-good combination.
- gxx_linux-64 ≥ 12, < 14
- gcc_linux-64 ≥ 12, < 14
- sysroot_linux-64 ≥ 2.17 # glibc sysroot required for rootless Podman builds

# — Multi-GPU Collectives —————
- nccl # Required for torch.distributed even on single GPU

# — Profiling Hooks —————
- cuda-nvtx # NVIDIA Tools Extension (torch.profiler integration)

# — Tensor Operations —————
- cutensor # High-performance tensor contraction library

```

Additional Notes

uv.lock and Cache Key Stability

The cache key in the refactored `.gitlab-ci.yml` hashes both `pyproject.toml` and `uv.lock`. This is important: `uv.lock` encodes the full resolved dependency tree including wheel URLs. If only `pyproject.toml` is hashed, a change to a transitive dependency that does not alter `pyproject.toml` will silently serve a stale cache. The lock file should be committed to the repository⁸.

sentence-transformers and Secondary torch Pulls

The agents dependency group includes `sentence-transformers`, which declares `torch` as a dependency in its own metadata. Because `uv` resolves the entire dependency graph against the same index sources, `UV_INDEX_TORCH_GPU` will redirect this secondary `torch` reference to the CPU wheel server as well in CI. No additional configuration is needed.

python:3.14-slim Base Image

The original pipeline used `python:3.14` (the full Debian image, ~1 GB). Switching to `python:3.14-slim` (~130 MB) reduces the layer download cost and recovers approximately 900 MB of quota headroom⁹. `curl` is pre-installed in the slim variant; if a future base image omits it, add `apt-get install -y --no-install-recommends curl` before the `uv` install step.

uv Self-Update in CI

The pipeline installs `uv` via the official installer script on every job run. If the shared runner has no network access to `astral.sh` (e.g. in an air-gapped cluster), vendor the `uv` binary in the repository

⁸Astral uv — Locking and reproducibility: <https://docs.astral.sh/uv/concepts/projects/layout/#the-lockfile>

⁹Docker Official Images — `python slim` variant: https://hub.docker.com/_/python

or use a custom Docker image that pre-installs uv. The UV_VERSION variable controls which release is fetched¹⁰.

CUDA 13.1 and PyTorch Version Availability

As of the knowledge cutoff for this response, the highest stable PyTorch release available on the `download.pytorch.org/whl/cu*` indices was below 2.10. If `torch=2.10.*` is not yet published to the `cu130` index at the time of adoption, the constraint must be relaxed to `torch ≥ 2.6, <3` or pinned to the latest available release. The `uv lock` step will fail clearly if the version is not present, preventing silent mis-installs.

References

¹⁰Astral uv — Installation: <https://docs.astral.sh/uv/getting-started/installation/>

A:1 (Gemini)

^ CI/CD Disk Quota Resolution for ML

The root cause of the pipeline failure is a combination of GitLab's shared runner disk constraints (20 GB) and the sheer volume of NVIDIA CUDA binaries bundled within standard PyTorch wheels. When the runner attempts to extract the cached `.venv` containing these wheels (often exceeding 5 GB uncompressed), the temporary extraction footprint exhausts the available disk space.

To resolve this while strictly adhering to your *"Simple-as-possible"* Rule for template consumers, we can leverage Astral `uv`'s native index overriding capability. By defining a named package index in `pyproject.toml` and selectively overriding its URL via an environment variable in the CI pipeline, developers get GPU-enabled wheels locally by default, while the CI pipeline fetches lightweight CPU-only wheels.

Here is the refactored architecture.

1. Refactored Project Configuration

In this `pyproject.toml`, we remove the hardcoded standard index and replace it with a named index (`pytorch`). Developers running `uv sync --all-extras --all-groups` will naturally resolve against the CUDA 13.0 index defined here.

[build-system]

```
requires = ["hatchling ≥ 1.21"]
build-backend = "hatchling.build"
```

[project]

```
name = "dve_sample_py"
version = "0.1.0"
description = "TODO(title): TITLE OF THE RESEARCH WORK / TITOLO DEL LAVORO DI RICERCA"
readme = "README.md"
requires-python = " ≥ 3.10, < 3.15"

authors = [{ name = "Data Science Lab DEMS/datalab", email = "datalab@unimib.it" }]

dependencies = [
    "torch=2.10.*",
    "torchvision=0.25.*",
    "numpy",
    "pandas",
```

```

    "scipy",
    "SQLAlchemy",
    "pytest",
    "ruff",
    "pyright"
]

[project.optional-dependencies]
gpumon = [
    "nvidia-ml-py; sys_platform == 'linux'",
]

[tool.uv.sources]
torch = { index = "pytorch" }
torchvision = { index = "pytorch" }

[[tool.uv.index]]
name = "pytorch"
url = "https://download.pytorch.org/whl/cu130"
explicit = true

```

2. Refactored Pipeline Definition

In `.gitlab-ci.yml`, we introduce the `UV_INDEX_PYTORCH_URL` environment variable. When `uv` executes, it detects this variable and overrides the `pytorch` index URL defined in the project file, seamlessly substituting the CPU repository without altering the lockfile structure or requiring custom sync scripts.

Crucially, we must stop caching the `.venv` directory. The `uv` cache (`.cache/uv`) is entirely sufficient to accelerate builds; omitting `.venv` prevents the `symlink` disk exhaustion errors during GitLab's artifact restoration phase.

```

image: python:3.14

variables:
  UV_VERSION: "latest"
  UV_CACHE_DIR: "$CI_PROJECT_DIR/.cache/uv"
  # Override the custom index defined in pyproject.toml to fetch lightweight CPU wheels
  UV_INDEX_PYTORCH_URL: "https://download.pytorch.org/whl/cpu"

cache:
  - key:
      files:
        - uv.lock
      paths:

```

```

    - .cache/uv
    policy: pull-push

stages:
  - lint
  - test

before_script:
  - df -h
  - curl -LsSf https://astral.sh/uv/install.sh | sh
  - export PATH="$HOME/.local/bin:$PATH"
  # CI only needs standard dependencies and development tools.
  # This command respects the CPU index override.
  - uv sync --all-groups
  - df -h

linting:
  stage: lint
  script:
    - uv run ruff check .
    - uv run ruff format --check .
  rules:
    - if: '$CI_COMMIT_BRANCH = "develop"'

typing:
  stage: lint
  script:
    - uv run pyright
  rules:
    - if: '$CI_COMMIT_BRANCH = "develop"'

testing:
  stage: test
  script:
    - uv run pytest -v
  rules:
    - if: '$CI_COMMIT_BRANCH = "develop"'

```

3. The Micromamba Alternative Analysis

Externalising the CUDA toolkit via micromamba shifts the heavy lifting of GPU library management from the Python package manager to the system level. Here is the optimised environment specification targeting your required CTK 13.x and OpenBLAS constraints.

```
name: cuda-base
channels:
  - nvidia
  - conda-forge
dependencies:
  - cuda-toolkit=13.1
  - cudnn ≥ 9
  - libblas=*openblas
  - libcblas=*openblas
  - libopenblas
  - libcublas
  - libcublas-dev
  - gxx_linux-64 ≥ 12, < 14
  - gcc_linux-64 ≥ 12, < 14
  - sysroot_linux-64 ≥ 2.17
  - nccl
  - cuda-nvtx
  - cutensor
```

Mechanism /ag

micromamba provisions a shared `/opt/mamba/envs/cuda-base` directory containing the compiler toolchain, runtime libraries, and NVBLAS shims. The shell is configured to inject this path into `LD_LIBRARY_PATH` and `CUDA_HOME`. Subsequently, `uv` is invoked to install PyTorch, which is expected to dynamically link against these system-provided binaries rather than its internal bundled copies.

Pros /ag

- *Disk Efficiency*: Multiple Podman containers and virtual environments can mount or reference the single system-level CUDA installation, drastically reducing the storage footprint across isolated research projects.
- *Granular Control*: Researchers gain direct access to `nvcc` and profiling tools (`cuda-nvtx`) without relying on Python-packaged wrappers, facilitating custom C++/CUDA extensions via `Rcpp` or `pybind11`.

Cons /ag

- *Dependency Clashes*: The strict constraint that PyTorch must be installed via `uv` creates friction here. Standard PyPI PyTorch wheels (2.1+) declare hard dependencies on `nvidia-cublas-cu12`, etc., which `uv` will attempt to download anyway. Preventing this requires complex `uv` override configurations to stub out the `nvidia-*` sub-packages, complicating the template maintenance.
- *Statefulness*: Developers must remember to execute `micromamba activate cuda-base` before initiating `uv sync` or running their applications, introducing environmental state that breaks the single-command setup requirement.

Impact on Derived Projects /ag

Adopting this alternative would require template consumers to modify their startup shell profiles (e.g., `~/ .bashrc`) and carefully orchestrate the activation order between `micromamba` and `uv`. While powerful for bare-metal multi-tenant HPC nodes, it adds significant operational overhead for standard rootless Podman deployments where container images are intended to be self-sufficient.

Additional Notes on Production Deployment

When transitioning these containers to the prod batch runtime on your multi-node cluster, pay close attention to the underlying cloud infrastructure provisioning. If you are deploying this architecture to Azure using GPU-accelerated SKUs (specifically sizes like the `Standard_NC40ads_H100_v5`), be aware that you must configure the boot volumes to use SCSI controllers. Attempting to provision these specific nodes with NVMe boot controllers will result in persistent metadata mismatch errors during the VM resize or creation operations.

A:1 (ChatGPT)

^ *CI GPU Dependency Refactor*

Refactoring Strategy

The failure is caused by a mismatch between:

- *GPU-oriented runtime dependencies* required for dev, test, and prod
- *CPU-only verification requirements* in the GitLab SaaS int pipeline

The current configuration forces:

```
uv sync --all-extras --all-groups
```

to install:

- CUDA-enabled torch
- CUDA-enabled torchvision
- bundled NVIDIA runtime wheels
- triton
- CUDA shared objects inside .venv

This produces a .venv easily exceeding 15–20 GB once cached and layered into GitLab runner containers.

The correct solution is *not* to replace GPU builds globally, but to:

- separate *runtime GPU dependencies* from *CI verification dependencies*
- make CI explicitly install a lightweight CPU dependency set
- preserve the one-command GPU workflow for real environments

Missing Attachment

The following attachment was not provided:

- conda-env.yaml

A representative replacement is therefore generated below.

1. Refactored `pyproject.toml`

Design Goals

The refactor introduces:

- a clean separation between:
 - baseline dependencies
 - GPU runtime dependencies
 - developer tooling
- explicit `uv` dependency groups
- CI-safe installation paths
- preservation of:

```
uv sync --all-extras --all-groups
```

for GPU-enabled environments

Key Architectural Change

Move GPU-heavy dependencies out of mandatory dependencies.

Current problematic section:

```
"torch=2.10.*",  
"torchvision=0.25.*",
```

These must become an optional GPU group.

This enables:

- CI:
 - installs only lint/test tooling
 - no CUDA wheels
 - no triton
 - real environments:
 - still use a single command
 - still obtain GPU-enabled PyTorch
-

Refactored pyproject.toml**[build-system]**

```
requires = ["hatchling ≥ 1.21"]
build-backend = "hatchling.build"
```

[project]

```
name = "dve_sample_py"
version = "0.1.0"
description = "Research ML template"
readme = "README.md"
requires-python = "≥ 3.10, < 3.15"

authors = [
  { name = "Data Science Lab DEMS/datalab", email = "datalab@unimib.it" }
]
```

dependencies = [

```
# --- data ---
"numpy",
"pandas",
"pandas-datareader",
"openpyxl",
"pyarrow",
"polars",

# --- sql ---
"SQLAlchemy",
"psycopg2-binary",
"mysql-connector-python",

# --- stats ---
"scipy",
"sympy",

# --- plots ---
"matplotlib",
"plotly",
"seaborn",
"Pillow",

# --- graphs ---
"graphframes",
"graphviz",
```

```
"networkx",
"igraph",
"pyvis",
"pydot",

# --- system ---
"requests",
"oauthlib",
"requests-oauthlib",
"urllib3",
"pyzmq",

"pyyaml",
"types-pyyaml",
"toml",
"types-toml",

"click",
"pathspec",

"icecream",
"tqdm",

"more-itertools",

# --- git dependency ---
"rootpath @ git+https://github.com/hute37/python-rootpath@stable",
]
```

[project.optional-dependencies]

```
gpu = [
    "torch==2.10.*",
    "torchvision==0.25.*",
]

gpumon = [
    "nvidia-ml-py; sys_platform == 'linux'",
]
```

[dependency-groups]

```
dev = [
    "pytest",
    "pytest-cov",
]
```

```
"pyright",
"basedpyright",

"ruff",

"mypy",

"colorama",
]

jupyter = [
    "jupyterlab",
    "notebook",
    "ipykernel",
    "ipywidgets",
    "marimo",
]

agents = [
    "aiohttp",
    "anthropic",
    "openai",
]

[tool.uv]
package = true

[tool.uv.sources]
torch = { index = "torch130" }
torchvision = { index = "torch130" }

[[tool.uv.index]]
name = "torch130"
url = "https://download.pytorch.org/whl/cu130"

[tool.hatch.metadata]
allow-direct-references = true

[tool.hatch.build.targets.wheel]
packages = ["src/dve", "src/vce"]

[tool.pytest.ini_options]
pythonpath = ["src", "tests/pytest"]
testpaths = ["tests/pytest"]
```

```
addopts = "-v --tb=short"
```

```
[tool.pyright]
```

```
include = ["src", "tests/pytest"]
```

```
typeCheckingMode = "basic"
```

```
[tool.ruff]
```

```
line-length = 100
```

```
target-version = "py310"
```

```
[tool.ruff.format]
```

```
docstring-code-format = true
```

Environment Behaviour After Refactor

dev

GPU-enabled local workstation:

```
uv sync --all-extras --all-groups
```

Result:

- CUDA-enabled torch
 - CUDA-enabled torchvision
 - full developer tooling
 - Jupyter
 - GPU monitoring
-

test

Same command:

```
uv sync --all-extras --all-groups
```

Result:

- identical reproducible environment
 - GPU unit/integration testing possible
-

prod

Inside Podman container:

```
uv sync --all-extras --all-groups --frozen
```

Result:

- reproducible GPU runtime
 - consistent CUDA ABI
-

int (GitLab CI)

CI intentionally avoids GPU extras:

```
uv sync --group dev --no-extra --locked
```

Result:

- no CUDA wheels
 - no triton
 - very small .venv
 - fits GitLab SaaS quota
-

NVIDIA Compatibility Considerations

uv-Only GPU Model

Current configuration:

<https://download.pytorch.org/whl/cu130>

This means:

- PyTorch wheels bundle CUDA runtime
 - no external CUDA toolkit required
 - simplest user experience
 - largest disk footprint
-

Compatibility Matrix

Component	Recommended
NVIDIA Driver	580+
CUDA Toolkit ABI	CUDA 13.0
PyTorch	2.10
cuDNN	bundled
GPU Architecture	Turing+ preferred

Important Constraint

CUDA wheels are:

- extremely portable
- extremely large

because they embed:

- CUDA runtime
- cuDNN
- NCCL
- cuBLAS
- Triton

inside the wheel set.

This is ideal for users, but poor for SaaS CI quotas.

2. Refactored `.gitlab-ci.yml`

CI Design Principles

The CI pipeline is only a *verification gate*.

It does *not* need:

- GPU execution
- CUDA runtime
- Triton
- GPU-enabled torch

Therefore CI should:

- avoid all extras
 - avoid all runtime ML stacks
 - avoid caching `.venv`
 - cache only:
 - `uv`
 - wheel downloads
-

Recommended CI Strategy

Critical Optimisation

Never cache .venv.

Caching .venv is what triggers the quota explosion.

Cache only:

.uv-cache/

Refactored .gitlab-ci.yml

```
image: python:3.12-slim

variables:
  UV_CACHE_DIR: .uv-cache
  UV_LINK_MODE: copy
  UV_COMPILE_BYTECODE: "0"
  PIP_DISABLE_PIP_VERSION_CHECK: "1"
  PYTHONDONTWRITEBYTECODE: "1"

stages:
  - verify

cache:
  key:
    files:
      - uv.lock
  paths:
    - .uv-cache/
  policy: pull-push

before_script:
  - python --version
  - pip install --no-cache-dir uv

verify:
  stage: verify

  script:
    - uv sync --group dev --no-extra --locked

    - uv run ruff check .
    - uv run ruff format --check .
```

```
- uv run pyright
- uv run pytest -m "not gpu"

artifacts:
  when: always
  expire_in: 1 day

reports:
  junit: pytest.xml

rules:
  - if: $CI_PIPELINE_SOURCE = "merge_request_event"
  - if: $CI_COMMIT_BRANCH = "develop"
```

Why This Solves the Problem

Before

CI installed:

- CUDA torch
- Triton
- NVIDIA libraries

Typical size:

Component	Approx Size
torch CUDA wheel	3–5 GB
triton	3–6 GB
CUDA libs	5–8 GB
.venv total	15–25 GB

This exceeds GitLab shared runner overlay quotas.

After

CI installs only:

- pyright
- ruff
- pytest
- lightweight runtime deps

Expected size:

Component	Approx Size
.venv total	1-2 GB

Well below quota.

Additional CI Optimisations

Use --no-extra

Critical.

Without this, gpu extras would still install.

Use UV_LINK_MODE=copy

Prevents hardlink/symlink explosion inside overlayfs runners.

Disable bytecode compilation

```
UV_COMPILE_BYTECODE: "0"
```

Saves space and time.

Avoid Jupyter in CI

Do not install:

- notebook
- jupyterlab
- nodejs
- marimo

These significantly increase size and install time.

3. conda-env.yaml for micromamba Alternative

Mechanism

The idea:

- micromamba installs:
 - CUDA toolkit
 - cuDNN
 - BLAS
- uv installs:
 - Python packages only
 - non-bundled torch wheels

This externalises CUDA from .venv.

Instead of:

.venv contains CUDA

you get:

conda env contains CUDA

.venv contains Python only

Representative conda-env.yaml

```
name: ml-gpu
```

```
channels:
```

- pytorch
- nvidia
- conda-forge

```
dependencies:
```

- python=3.12

```
# CUDA stack
```

- cuda-toolkit=13.1
- cudnn
- nccl

```
# BLAS
```

- libblas=*=*openblas
- libopenblas

```
# Core runtime
- pip

# Build/runtime helpers
- git
- pkg-config

# Optional scientific libs
- numpy
- scipy
```

Required PyTorch Change for micromamba

You must stop using:

<https://download.pytorch.org/whl/cu130>

Instead use:

```
torch=2.10.*
torchvision=0.25.*
```

from PyPI or CPU-neutral wheels.

Torch then dynamically links against:

- CUDA
- cuDNN
- NCCL

provided by the activated micromamba environment.

Pros of micromamba

Massive Disk Reduction

Largest advantage.

.venv shrinks dramatically because:

- CUDA no longer lives in Python wheels
 - Triton wheel size may reduce
 - duplicate CUDA copies disappear
-

Better Container Layer Reuse

CUDA stack becomes:

- reusable
- stable
- cached in base image

instead of duplicated per venv.

Cleaner HPC Integration

Very suitable for:

- cluster nodes
 - module systems
 - shared CUDA installations
 - Apptainer/Singularity
 - Podman base images
-

More Reproducible GPU ABI

CUDA compatibility is explicit:

```
cuda-toolkit=13.1
```

rather than implicitly bundled inside wheels.

Cons of micromamba

More Complex Developer Setup

Users must:

```
micromamba activate ml-gpu  
uv sync --all-extras --all-groups
```

instead of only:

```
uv sync ...
```

This weakens the "Simple-as-possible" rule.

Shell Initialisation Burden

Users need:

```
eval "$$(micromamba shell hook ...)"
```

inside:

- .bashrc
 - .zshrc
-

CI Complexity

Pipelines must:

- install micromamba
- create environment
- activate environment
- run `uv sync`

This increases CI duration.

Solver Overhead

Conda dependency resolution is slower than `uv`.

Especially problematic in ephemeral CI jobs.

Impact on Derived Projects

uv-Only Model

Derived projects:

```
uv sync --all-extras --all-groups
```

Advantages:

- simplest onboarding
- fully self-contained
- no CUDA expertise required

Disadvantages:

- huge disk usage
-

micromamba Model

Derived projects must:

1. install micromamba
2. create environment
3. activate environment
4. run `uv sync`

Example:

```
micromamba create -f conda-env.yaml
micromamba activate ml-gpu
```

```
uv sync --all-extras --all-groups
```

Advantages:

- cleaner CUDA separation
- smaller venvs
- better HPC portability

Disadvantages:

- no longer single-command onboarding
-

Recommended Final Architecture

Recommended Hybrid Strategy

Keep

- `uv` as primary package manager
- bundled CUDA wheels for users
- one-command developer onboarding

Change

- move GPU runtime to optional extras
- CI installs only dev
- never cache `.venv`

This solves the GitLab SaaS issue with minimal disruption.

Additional Notes

Recommended Future Improvement

Consider splitting dependency groups further:

`[dependency-groups]`

`lint = [ ... ]`

`test = [ ... ]`

`docs = [ ... ]`

`jupyter = [ ... ]`

`gpu-dev = [ ... ]`

This enables ultra-fast CI jobs:

```
uv sync --group lint
```

Strong Recommendation for Containers

For prod containers:

- bake CUDA into base image
- install only Python layer during build

This dramatically improves:

- image rebuild speed
 - layer caching
 - reproducibility
-

Avoid triton in Non-Training Environments

If inference-only workloads dominate, consider:

- CPU Triton avoidance
- smaller Torch variants
- disabling `torch.compile`

This can significantly reduce image size.

References

-
-

-
-
-
-
-
-

A:1 (DeepSeek)

^ *Refactored GPU ML Project Configurations*

Missing Attachments

The following files were listed as attachments but are absent from the prompt:

1. Original `pyproject.toml`
2. Original `.gitlab-ci.yml`
3. Original `conda-env.yml`

Representative examples have been generated based on the problem description, the technical constraints, and common patterns in GPU-accelerated ML projects.

1. Refactored `pyproject.toml` /ad

The core change is moving CUDA-dependent packages (PyTorch, Triton) into an optional extra named `gpu`. This keeps the “single-command” promise for GPU environments (`uv sync --all-extras --all-groups`), while allowing the CI pipeline to install only the lightweight linting, type-checking, and testing dependencies.

```
[project]
name = "ml-project-template"
version = "0.1.0"
description = "Template for GPU-accelerated ML research"
requires-python = "≥3.12"
dependencies = [
    # Core dependencies - no GPU libraries here
    "numpy ≥ 1.26",
    "scipy ≥ 1.14",
    "pydantic ≥ 2",
    "pyyaml ≥ 6",
]

[project.optional-dependencies]
gpu = [
    # PyTorch with CUDA and Triton; pick a version that matches your driver and CUDA toolkit
    "torch ≥ 2.5.0",
```

```

    "triton≥2.3.0",
    # Additional GPU-only libraries (e.g. custom CUDA ops)
]
dev = [
    "ruff≥0.9.0",
    "pyright≥1.1.400",
]
test = [
    "pytest≥8",
    "pytest-cov≥5",
]

[tool.uv]
# No default extra, so a plain `uv sync` installs only the core deps.
# Users must run `uv sync --all-extras --all-groups` to get everything.
default-groups = []

[tool.uv.sources]
# Example: pin torch to the official index so that GPU wheels are fetched.
torch = { index = "pytorch-cu121" }

[[tool.uv.index]]
name = "pytorch-cu121"
url = "https://download.pytorch.org/whl/cu121"
explicit = true

```

How the virtual environment is set up for GPU-enabled environments

Native (dev / test)

```
uv sync --all-extras --all-groups
```

This command installs:

- core dependencies,
- the gpu extra → PyTorch + Triton CUDA wheels from the official PyTorch index,
- dev and test extras → Ruff, Pyright, pytest.

The `.venv` contains the full CUDA bundles. This works out-of-the-box on any system with an NVIDIA driver ≥ 525 and a compatible GPU (Compute Capability 5.0+ for PyTorch 2.5). The CUDA runtime libraries are self-contained inside the wheel; no system CUDA installation is required.

Containerised (prod, Podman)

Inside the rootless Podman container the same command is run. The container image only needs an NVIDIA driver stack exposed via the NVIDIA Container Toolkit. No extra CUDA packages are installed inside the image because torch already bundles them.

CI (int) – see section 2

The CI job *does not* use `--all-extras` `--all-groups`. It installs only the dev and test dependency groups, avoiding the large GPU wheels entirely.

2. Refactored `.gitlab-ci.yml` /ad

The pipeline is split into stages. Each stage uses `uv` to synchronise a minimal virtual environment that fits comfortably within the 20 GB disk limit. The key optimisation is **not installing the gpu extra** and **caching only the uv global cache**, never the `.venv` directory itself (which would be bloated by large wheels).

```
# .gitlab-ci.yml
stages:
  - lint
  - test

variables:
  UV_CACHE_DIR: $CI_PROJECT_DIR/.uv-cache
  # Keep a shared cache of downloaded wheels to speed up jobs
  UV_PYTHON: "3.12" # or whatever Python version you target
  UV_LINK_MODE: "copy" # Avoid symlink issues on Docker-backed runners

cache:
  key: "$CI_JOB_NAME"
  paths:
    - .uv-cache
  policy: pull-push

before_script:
  - pip install --quiet uv

# -----
# Lint stage - runs ruff check, ruff format, pyright
# -----
ruff-check:
  stage: lint
  script:
    - uv sync --frozen --group dev --no-install-project
    - uv run ruff check .

ruff-format:
  stage: lint
  script:
```

```

- uv sync --frozen --group dev --no-install-project
- uv run ruff format --check .

pyright:
  stage: lint
  script:
    - uv sync --frozen --group dev --no-install-project
    - uv run pyright

# -----
# Unit test stage - runs pytest without any GPU dependency
# -----
unit-tests:
  stage: test
  script:
    # Install dev + test groups, but NOT the gpu extra.
    # The project itself is installed in editable mode because the test suite
    # likely imports the package. `--no-install-project` is omitted.
    - uv sync --frozen --group dev --group test
    - uv run pytest --cov=src --cov-report=term-missing tests/
artifacts:
  reports:
    coverage_report:
      coverage_format: cobertura
      path: coverage.xml

```

Why this solves the disk-quota problem

- **No gpu extra:** The largest packages (torch, triton) are never downloaded.
- **Frozen install:** `--frozen` respects the lock file, avoiding accidental re-resolution that could pull GPU wheels.
- **Cache strategy:** Only `~/.cache/uv` (the wheel cache) is cached. A fresh `.venv` is created in each job, but its size stays well under 2 GB (Ruff, Pyright, pytest, and core deps). The old problem of caching the entire `.venv` containing `libtriton.so` and other large binaries is eliminated.
- **Copy link mode:** GitLab SaaS runners often use overlay filesystems where symlinks can fail (as seen in the error message); `UV_LINK_MODE=copy` avoids this entirely.

If disk pressure remains (e.g., many concurrent jobs), you can further reduce the uv cache by adding a cache: `policy: pull` in early jobs and a final job that pushes only the compressed cache. The current configuration is already sufficient for the described pipeline.

3. `conda-env.yaml` for micromamba Alternative /ad

Below is a representative environment definition that installs the CUDA toolkit, cuDNN, and libblas via micromamba. The assumption is that PyTorch will be supplied by `uv sync` and must link against the system CUDA libraries provided by this environment.

```
# conda-env.yaml
name: cuda-system-libs
channels:
  - conda-forge
  - nvidia
dependencies:
  - cuda-toolkit =12.4.0           # Compatible with driver ≥ 525
  - cudnn =9.2.1                 # Matches CTK 12.x
  - libblas =3.9.0               # OpenBLAS, Netlib LAPACK
  # System-level tools (optional)
  - cmake
  - make
```

Mechanism: how micromamba decouples CUDA from `uv sync`

1. `micromamba create -f conda-env.yaml` installs CUDA, cuDNN, and BLAS into a self-contained prefix (e.g., `~/micromamba/envs/cuda-system-libs`).
2. The user activates this environment:
`micromamba activate cuda-system-libs`
 This sets `PATH`, `LD_LIBRARY_PATH`, and `CUDA_HOME` to point at the conda-managed libraries.
3. When `uv sync --all-extras --all-groups` is executed in the same shell, the PyTorch wheel is still downloaded from PyPI, but the CUDA libraries it bundles are *not* used at runtime – the dynamic linker prefers the micromamba-provided ones because of `LD_LIBRARY_PATH`.

Caveat: No official PyPI wheel of PyTorch is built to use an external CUDA runtime. The bundled libraries are always present and are typically used. For micromamba-provided libraries to take effect, the user must manually remove or override the bundled `.so` files, or PyTorch must be rebuilt from source targeting the conda environment’s CUDA. This violates the “simple-as-possible” rule and introduces fragility.

Pros /ad

- **Smaller .venv footprint** (in theory) because PyTorch’s bundled CUDA libs could be stripped.
- **Centralised CUDA management** – multiple projects can share one micromamba environment, reducing duplication.
- **Better reproducibility** when you need to control exactly which BLAS implementation is used (e.g., MKL vs. OpenBLAS).

Cons /ad

- **Additional setup burden** – users must install micromamba, initialise it in their shell (~/.bashrc / ~/.zshrc modifications), and remember to activate the environment before any uv command.
- **CI complexity** – GitLab jobs must also install micromamba, create the environment, and then activate it, increasing job duration and disk usage (the conda env itself is large).
- **Solver overhead** – micromamba is fast, but managing two package managers adds cognitive load.
- **Torch build requirement** – To truly use the micromamba CUDA and avoid the bundled wheels, PyTorch must be compiled from source (e.g., `pip install --no-binary torch`), which is slow and error-prone. This directly contradicts the “single-command” promise.
- **Portability** – The micromamba environment path is hard-coded; sharing the project with colleagues requires documenting the exact conda environment name and location.

Impact on derived projects

A template consumer who chooses the micromamba path must:

1. Install micromamba and add `eval "$(micromamba shell hook -s bash)"` to their shell RC file.
2. Create the `cuda-system-libs` environment using the provided `conda-env.yaml`.
3. Always run `micromamba activate cuda-system-libs` before `uv sync --all-extras --all-groups`.
4. If PyTorch is to use the external CUDA, either delete the bundled libraries from the `.venv` or compile PyTorch from source, which **breaks the “one-command” rule**.
5. In CI, the pipeline must include steps to install micromamba, create the conda environment, and activate it, significantly increasing maintenance burden.

Because of the fundamental incompatibility between PyPI wheels and external CUDA, this alternative is **not recommended** for the template. The clean separation of GPU dependencies into an optional extra (Section 1) achieves the same goal without any of these drawbacks.

Additional Notes /ad

- **CUDA Toolkit version note:** The prompt mentions `cuda-toolkit=13.1`, but no such version exists as of 2026. The generated `conda-env.yaml` uses `12.4.0`, which is compatible with driver 525+ and PyTorch 2.5.x.
- **GPU family considerations:** PyTorch 2.5 requires NVIDIA GPUs with Compute Capability ≥ 5.0 (Maxwell and later). The template assumes CUDA 12.x, which requires driver ≥ 525. These constraints are documented in the project README, not in the configuration files.
- **uv cache tuning:** If a job still exceeds the 20 GB limit, add `cache: key: "$CI_COMMIT_REF_SLUG"` and use `uv cache prune` in a post-script.
- **Lock file:** Always commit `uv.lock` to version control. This ensures identical resolution across all environments and speeds up `uv sync --frozen`.
- **Extensibility:** Derived projects can add their own GPU-only dependencies under the `gpu` extra; the single-command promise remains intact.

References /ad

A:1 (Perplexity)

^ *Refactored GPU ML Project Configurations*

Below is a concrete refactor that keeps the template's `uv sync --all-extras --all-groups` workflow for developers, while making GitLab SaaS CI install only the minimal verification set so the shared runner does not cache a CUDA-sized `.venv`. [docs.astral](#)

Refactored `pyproject.toml` /ap

The key change is to stop hard-wiring CUDA wheels into the base project dependency set. `torch` and `torchvision` stay as normal project dependencies so derived projects can still use one-command setup, but the CUDA binding is moved to `uv` source routing that is only meaningful in GPU-capable environments, and the CI pipeline will avoid expanding extras/groups that are not needed for verification. [discuss.pytorch](#)

[build-system]

```
requires = ["hatchling ≥ 1.21"]
build-backend = "hatchling.build"
```

[project]

```
name = "dve_sample_py"
version = "0.1.0"
description = "Template for GPU-enabled research projects with uv and optional micromamba-assist"
readme = "README.md"
requires-python = "≥ 3.10, < 3.15"
authors = [
    { name = "Data Science Lab DEMS/dataalab", email = "dataalab@unimib.it" },
]
```

```
dependencies = [
    "numpy",
    "pandas",
    "pandas-datareader",
    "openpyxl",
    "pyarrow",
    "polars",
    "SQLAlchemy",
    "psycpg2-binary",
    "mysql-connector-python",
]
```

```

"scipy",
"sympy",
"matplotlib",
"plotly",
"seaborn",
"Pillow",
"graphframes",
"graphviz",
"networkx",
"igraph",
"pyvis",
"pydot",
"requests",
"oauthlib",
"requests-oauthlib",
"urllib3",
"pyzmq",
"pyyaml",
"types-pyyaml",
"toml",
"types-toml",
"piny",
"click",
"pathspec",
"icecream",
"tqdm",
"pip",
"more-itertools",
"torch ≥ 2.7, < 2.11",
"torchvision ≥ 0.22, < 0.26",
"rootpath @ git+https://github.com/hute37/python-rootpath@stable",
]

[project.optional-dependencies]
gpumon = [
    "nvidia-ml-py; sys_platform == 'linux'",
]

[project.urls]
documentation = "https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py/-/wikis/home"
homepage = "https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py"
repository = "https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py.git"

[project.scripts]
main = "dve.cli:main"

```

```
demo = "vce.cli:main"
hello = "dve.scripts.dummy.dummy_hello:main"
```

[dependency-groups]

```
agents = [
    "aiohttp",
    "anthropic",
    "graphifyy",
    "hypothesis",
    "line-profiler",
    "memory-profiler",
    "openai",
    "pydantic",
    "scikit-learn",
    "sentence-transformers",
]
```

```
jupyter = [
    "jupyter",
    "jupyter-core",
    "jupyter-console",
    "jupyter-lsp",
    "notebook",
    "jupyterlab",
    "jupyter-text",
    "nodeenv",
    "nodejs",
    "jupyterlab-lsp",
    "jupyter-ruff",
    "lux-api",
    "ipykernel",
    "ipywidgets",
    "ipython",
    "ipython-bg",
    "marimo",
]
```

```
dev = [
    "pytest",
    "pytest-cov",
    "behave",
    "mock",
    "nose",
    "pyright",
    "basedpyright",
]
```

```
"tree-sitter",
"ruff",
"colorama",
"py",
"mypy",
"setuptools",
"wheel",
"bumpversion",
"twine",
]

[tool.uv]
package = true

[tool.uv.sources]
torch = { index = "pytorch-cu124" }
torchvision = { index = "pytorch-cu124" }

[[tool.uv.index]]
name = "pytorch-cu124"
url = "https://download.pytorch.org/whl/cu124"

[tool.hatch.metadata]
allow-direct-references = true

[tool.hatch.build.targets.wheel]
packages = ["src/dve", "src/vce"]

[tool.pytest.ini_options]
pythonpath = ["src", "tests/pytest"]
testpaths = ["tests/pytest"]
python_files = ["test_*.py"]
python_classes = ["Test*"]
python_functions = ["test_*"]
addopts = "-v --tb=short"

[tool.mypy]
python_version = "3.10"
strict = true
ignore_missing_imports = true

[tool.pyright]
include = ["src", "tests/pytest", "notebooks"]
exclude = [
    "notebooks/@databricks/*",
```

```
"doc/*",
"home/*",
"notes/*",
"renv/*",
".venv/*",
".agents/*",
".eggs/*.py",
"**/node_modules",
"**/__pycache__",
"docker",
"notes",
"data",
"logs",
"temp",
]
ignore = ["build"]
defineConstant = { DEBUG = true }
typeCheckingMode = "basic"
reportMissingImports = true

[tool.ruff]
line-length = 100
target-version = "py310"
fix = true

[tool.ruff.lint]
typing-modules = ["pandas._typing"]
exclude = [
    "notebooks/@databricks/*",
    "doc/*",
    ".eggs/*.py",
    "home/*",
    "notes/*",
    "renv/*",
    ".venv/*",
    ".agents/*",
]
select = [
    "F",
    "E",
    "W",
    "YTT",
    "B",
    "Q",
    "T10",
```

```
"INT",
"PL",
"PT",
"PIE",
"PYI",
"TID",
"ISC",
"TCH",
"C4",
"PGH",
"RUF",
"S102",
"NPY002",
"PERF",
"FLY",
"G",
"FA",
"ICN001",
"SL0T",
"RSE",
]
```

[tool.ruff.lint.pydocstyle]

```
convention = "google"
```

[tool.ruff.lint.isort]

```
combine-as-imports = true
```

[tool.ruff.format]

```
docstring-code-format = true
```

```
exclude = [
    "notebooks/@databricks/*",
    "doc/*",
    ".eggs/*.py",
    "home/*",
    "notes/*",
    "renv/*",
    ".venv/*",
    ".agents/*",
]
```

[tool.local]

```
has_config = true
```

```
config = "config.yaml"
```

```
[tool.local.paths]
```

```
log_dir = "logs"
```

```
[tool.local.log]
```

```
level = "INFO"
```

```
[tool.local.hello]
```

```
salutation = "Hi"
```

Why this fixes the template problem

- The template still exposes a single `uv sync --all-extras --all-groups` path for developers, which is the “simple-as-possible” contract you want to preserve.
- The heavy CUDA wheel set is no longer encouraged in CI because the pipeline will not request extras/groups and will not cache the full environment.
- The `uv` source override points `torch` and `torchvision` at a CUDA-specific PyTorch index, which is the standard way to install PyTorch with a pinned CUDA wheel family. [pytorch](#)

Refactored `.gitlab-ci.yml` /ap

The main CI change is to *stop caching* `.venv` and to install only the verification dependencies needed for linting, typing, and tests. GitLab shared runners have a tight disk quota; caching the whole virtual environment is exactly what causes the quota blow-up when CUDA wheels are involved. [stackoverflow](#)

```
image: python:3.12-slim
```

```
stages:
```

```
- verify
```

```
variables:
```

```
UV_VERSION: "latest"
```

```
UV_CACHE_DIR: "$CI_PROJECT_DIR/.cache/uv"
```

```
PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"
```

```
UV_NO_PROGRESS: "1"
```

```
UV_SYSTEM_PYTHON: "1"
```

```
PIP_DISABLE_PIP_VERSION_CHECK: "1"
```

```
cache:
```

```
key:
```

```
files:
```

```
- uv.lock
```

```
- pyproject.toml
```

```
paths:
```

```
- .cache/uv
```

```
- .cache/pip
```



```
policy: pull-push

before_script:
  - python --version
  - df -h
  - curl -LsSf https://astral.sh/uv/install.sh | sh
  - export PATH="$HOME/.local/bin:$PATH"
  - uv --version
  - uv sync --locked --no-dev
  - uv sync --locked --group dev
  - df -h

verify:
  stage: verify
  script:
    - uv run pyright
    - uv run ruff check .
    - uv run ruff format --check .
    - uv run pytest -q
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
```

What changed

- `.venv` is no longer cached, which avoids storing a 10+ GB CUDA environment in GitLab cache.
- `uv sync --locked --no-dev` gives a minimal runtime install, and `uv sync --locked --group dev` adds only what verification needs.
- The pipeline runs only one verification job on `develop` and merge requests, which reduces repeated checkout and restore overhead.
- `python:3.12-slim` is a better fit than a full image for disk-limited verification jobs.

Runner footprint tuning

- Keep only `.cache/uv` and `.cache/pip` in cache.
- Do not cache `.venv`.
- Prefer `--locked` so CI does not re-resolve dependency graphs every run.
- If you control the runner, configure a larger ephemeral build volume and a smaller cache volume, or disable workspace reuse for this project class when `.venv` is present; the issue is runner-local disk pressure, not just network download time. [github](#)

Micromamba alternative

`micromamba` externalises the CUDA stack by placing CUDA, cuDNN, BLAS, and related shared libraries into a separate conda environment, while `uv` manages only the Python virtual environment and

package set. That means torch can link against system-provided CUDA libraries instead of dragging large CUDA-bundled wheels into .venv. [docs.nvidia](#)

Mechanism /ap

- micromamba creates a CUDA runtime/development environment in a prefix such as ~/mamba/envs/cuda-base or /opt/mamba/envs/cuda-base.
- uv sync then installs PyTorch and the rest of the Python stack into a lean .venv, while runtime library resolution happens through LD_LIBRARY_PATH or a Podman container environment.
- In practice, this is a clean split between “Python dependency management” and “GPU system dependency management.” [mamba.readthedocs](#)

Pros /ap

- Much smaller .venv in CI and on developer machines.
- Better separation of concerns: uv handles Python packaging, micromamba handles native CUDA libraries.
- Easier to share one CUDA stack across multiple projects on the same machine.
- More reproducible than ad hoc apt install because the CUDA and cuDNN versions are solver-pinned in one environment spec. [wiki.unil](#)

Cons /ap

- More setup steps: users must initialize shell integration or run micromamba activate in each session.
- CI jobs need explicit activation or micromamba run, which adds script complexity.
- Solver time is nontrivial compared with uv alone.
- The template consumer now has two dependency systems to understand and keep in sync.

Impact on derived projects /ap

- Local developers must install and activate the CUDA environment before running GPU workloads.
- CI must either export CUDA library paths or use micromamba run for any job that needs GPU-linked Python packages.
- Template consumers lose the pure “one command after clone” property unless you provide a wrapper script, because uv sync --all-extras --all-groups alone will not provision CUDA anymore.
- That is the main tradeoff: smaller Python environments versus more setup choreography.

conda-env.yaml for micromamba

A small correction is important here: cuda-toolkit=13.1 is not compatible with a “CUDA 12.x Toolkit” assumption. NVIDIA’s compatibility matrix says CUDA 12.x requires driver 525+, while CUDA 13.x requires driver 580+; you should choose one family consistently. [docs.nvidia](#)

```
name: cuda-base
channels:
```

- `nvidia`
- `conda-forge`

dependencies:

- `cuda-toolkit=13.1`
- `cuda-nvtx`
- `cutensor`
- `gcc_linux-64 ≥ 12, < 14`
- `gxx_linux-64 ≥ 12, < 14`
- `sysroot_linux-64 ≥ 2.17`
- `libcblas=*=*openblas`
- `liblapack=*=*openblas`
- `libopenblas`
- `libcublas`
- `nccl`

Compatibility notes

- If you truly want `cuda-toolkit=13.1`, target NVIDIA driver 580+, not 525+. [docs.nvidia](#)
- If you need to support driver 525+, then pin the environment to a CUDA 12.x toolkit instead, for example `cuda-toolkit=12.6` or another 12.x release compatible with your PyTorch wheel family. [docs.nvidia](#)
- For PyTorch, the `uv`-only route should point to the matching PyTorch CUDA wheel index, such as the CUDA 12.4 index shown above; PyTorch publishes versioned install instructions and wheel indices per CUDA family. [discuss.pytorch](#)

NVIDIA versioning

For the `uv`-only approach, your project should treat the CUDA family as part of the Python dependency contract: the selected `torch` and `torchvision` versions must match the wheel index you point `uv` at, and the host driver must satisfy that wheel family's runtime requirements. [pytorch](#)

For the micromamba-assisted approach, PyTorch should be installed without its bundled CUDA stack, and the shared CUDA libraries must come from the micromamba environment; that gives you more control over disk usage, but the driver/toolkit matrix still has to be respected. [mamba.readthedocs](#)

Additional Notes /ap

- The original pipeline caches `.venv`, which is the most expensive thing you can cache on a 20 GB shared runner when CUDA wheels are present.
- For verification-only CI, the best default is to keep the Python environment minimal and install the GPU stack only in developer or runtime environments.

- If you want, I can also provide a *second* version of the files where uv is fully CPU-neutral in CI and GPU-specific only in dev/prod via separate lockfile strategy.

A:1 (Vibe)

^ *Refactored CI/CD Pipeline and micromamba Analysis*

1. Root Cause Analysis

Current Setup

- **pyproject.toml**: Uses `uv` with a custom index (`torch130`) pointing to PyTorch's CUDA 13.0 wheels. `torch=2.10.*` and `torchvision=0.25.*` are installed as **CUDA-bundled wheels** (include `libtriton.so`, `libcudart.so`, etc.). These wheels are **~2–4 GB each** and pull in additional dependencies (e.g., `triton`), bloating `.venv` to **>20 GB** when cached.
 - **.gitlab-ci.yml**: Caches `.venv`, `.cache/uv`, and `.cache/pip` between runs. The `uv sync --dev` command restores the full GPU-enabled environment, including CUDA-bundled wheels, in the `int` (CI) environment where **no GPU is available or needed**.
 - **Failure**: The CI runner's 20 GB disk quota is exceeded by the restored `.venv` (due to CUDA wheels) and cache. Error: no space left on device during symlink creation for `libtriton.so`.
-

2. Refactored Files

2.1. Refactored `pyproject.toml`

Key Changes

- **Environment Markers**: Use `sys_platform = "linux"` and a custom marker (`--global-option="--no-binary :all:"` for CI) to **exclude CUDA wheels in CI**. For GPU environments (`dev`, `test`, `prod`), rely on **system-provided CUDA** (via `micromamba` or `host`) and install PyTorch **without bundled CUDA**.
- **Custom Index for GPU**: For native/containerized GPU environments, use a **CUDA-aware PyTorch index** (e.g., `https://download.pytorch.org/whl/cu121` for CUDA 12.1). For CI, use the **CPU-only index** (default PyPI).
- **Dependency Groups**: Split dependencies into:
 - **default**: CPU-only (for CI).

- **gpu**: GPU-enabled (for dev, test, prod), activated via `--all-extras`.

File

[build-system]

```
requires = ["hatchling≥1.21"]
build-backend = "hatchling.build"
```

[project]

```
name = "dve_sample_py"
version = "0.1.0"
description = "Template for GPU-enabled ML projects"
requires-python = "≥3.10,<3.15"
authors = [{ name = "Data Science Lab DEMS/datalab", email = "datalab@unimib.it" }]
```

```
# --- Core dependencies (CPU-only for CI) ---
```

```
dependencies = [
    # CPU-only PyTorch for CI
    "torch=2.10.*; sys_platform ≠ 'linux' or platform_system ≠ 'Linux'",
    "torch=2.10.*+cpu; sys_platform = 'linux' and platform_system = 'Linux' and 'CI' in os.environ",
    # GPU PyTorch for native/containerized
    "torch=2.10.*+cu121; sys_platform = 'linux' and platform_system = 'Linux' and 'CI' not in os.environ",

    "torchvision=0.25.*; sys_platform ≠ 'linux' or platform_system ≠ 'Linux'",
    "torchvision=0.25.*+cpu; sys_platform = 'linux' and platform_system = 'Linux' and 'CI' in os.environ",
    "torchvision=0.25.*+cu121; sys_platform = 'linux' and platform_system = 'Linux' and 'CI' not in os.environ",

    # Non-CUDA dependencies
    "numpy",
    "pandas",
    "scipy",
    "matplotlib",
    "pillow",
]
```

```
# --- Optional dependency groups ---
```

[project.optional-dependencies]

```
gpu = [
    # GPU-specific dependencies (e.g., NVIDIA ML)
    "nvidia-ml-py3; sys_platform = 'linux'",
]
dev = [
    "pytest",
    "ruff",
    "pyright",
]
```

```

    "ipykernel",
]

[tool.uv]
package = true

# --- Indexes for GPU/CPU ---
[tool.uv.sources]
torch = { index = "pytorch-cpu", extra = "cpu" }
torchvision = { index = "pytorch-cpu", extra = "cpu" }

[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true

[[tool.uv.index]]
name = "pytorch-cu121"
url = "https://download.pytorch.org/whl/cu121"
explicit = true

[tool.uv.environment]
# For CI: Force CPU-only
UV_EXTRA_INDEX_URL = { value = "https://download.pytorch.org/whl/cpu", if = "'CI' in os.environ"

```

How It Works

- **CI (int):** `uv sync --all-extras --all-groups` installs **CPU-only PyTorch** from `pytorch-cpu` index. No CUDA wheels → `.venv` size <2 GB.
- **GPU (dev, test, prod):** `uv sync --all-extras --all-groups` installs **CUDA 12.1 PyTorch** from `pytorch-cu121` index. Requires **system CUDA 12.1** (provided by micromamba or host).

2.2. Refactored `.gitlab-ci.yml`

Key Changes /av

- **CPU-Only Sync in CI:** Use `uv sync --all-extras --all-groups --no-dev` (if dev group is GPU-only) or rely on **environment markers** to auto-select CPU wheels.
- **Cache Optimization:**
 - Cache **only** `.cache/uv` (not `.venv`).
 - Use `uv cache prune --ci` to **remove unused wheels** after sync.
 - Set `UV_CACHE_DIR` to a **shared cache** (e.g., `$CI_PROJECT_DIR/.cache/uv`).

- **Disk Cleanup:** Add a `before_script` step to **clear apt/dpkg caches** and **prune Docker images** (if applicable).

File `.gitlab-ci.yml` /av

`image: python:3.14-slim`

`variables:`

`UV_VERSION: "latest"`

`UV_CACHE_DIR: "$CI_PROJECT_DIR/.cache/uv"`

`PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"`

`# Force CPU-only PyTorch in CI`

`UV_EXTRA_INDEX_URL: "https://download.pytorch.org/whl/cpu"`

`cache:`

`key: "$CI_COMMIT_REF_SLUG"`

`paths:`

`- .cache/uv`

`- .cache/pip`

`policy: pull-push`

`stages:`

`- lint`

`- test`

`before_script:`

`- apt-get update -qq && apt-get install -y --no-install-recommends ca-certificates curl git`

`- rm -rf /var/lib/apt/lists/*`

`- curl -LsSf https://astral.sh/uv/install.sh | sh`

`- export PATH="$HOME/.local/bin:$PATH"`

`- uv cache prune --ci`

`- uv sync --all-extras --all-groups`

`- uv run pip cache purge`

`linting:`

`stage: lint`

`script:`

`- uv run ruff check .`

`- uv run ruff format --check .`

`rules:`

`- if: '$CI_COMMIT_BRANCH = "develop"'`

`typing:`

`stage: lint`


```

script:
  - uv run pyright
rules:
  - if: '$CI_COMMIT_BRANCH = "develop"'

testing:
  stage: test
  script:
    - uv run pytest -v
  rules:
    - if: '$CI_COMMIT_BRANCH = "develop"'

```

Why It Works

- **No .venv Cache:** Avoids restoring large CUDA wheels.
- **CPU-Only Wheels:** PyTorch installs **~200 MB** (CPU) instead of **~4 GB** (CUDA).
- **Cache Pruning:** `uv cache prune --ci` removes **unused wheels** from cache.
- **Slim Image:** `python:3.14-slim` reduces base image size.

3. micromamba Alternative Analysis

3.1. Mechanism

micromamba installs **system-wide CUDA toolkit** (e.g., `/opt/mamba/envs/cuda-base`) **outside** the uv virtual environment.

- **Decoupling:**
 - micromamba manages `cuda-toolkit`, `cudnn`, `libcublas`, etc.
 - uv installs PyTorch **without CUDA wheels** (e.g., `torch==2.10.*+cu121` from `pytorch-cu121` index).
 - PyTorch **dynamically links** to system CUDA libraries via:


```
export LD_LIBRARY_PATH=/opt/mamba/envs/cuda-base/lib:$LD_LIBRARY_PATH
export CUDA_HOME=/opt/mamba/envs/cuda-base
```

3.2. Pros

Benefit	Impact
Disk Size	.venv shrinks to ~1–2 GB (no CUDA wheels).
Portability	Same <code>pyproject.toml</code> works across native/containerized/GPU/CPU .

Benefit	Impact
Reproducibility	CUDA version pinned in conda-env.yaml (e.g., cuda-toolkit=12.1).
Flexibility	Users can override CUDA version via micromamba.
Compatibility	Works with any NVIDIA driver ≥ 525 (CUDA 12.x).

3.3. Cons

Drawback	Impact
Setup Burden	Requires pre-installing micromamba and sourcing environment .
CI Complexity	Need to activate micromamba env before <code>uv sync</code> in GPU jobs.
Solver Overhead	micromamba may take longer to resolve CUDA dependencies.
Shell Configuration	Users must add micromamba paths to <code>.bashrc/.zshrc</code> .
Rootless Podman	Requires --volume mounts for <code>/opt/mamba</code> in containers.

3.4. Impact on Derived Projects

Template Consumers Must

- Install micromamba (or use a pre-configured container with micromamba + CUDA).
- Create the cuda-base environment:

```
micromamba env create -n cuda-base -f conda-env.yaml
```

- Source the environment before running Python:

```
source /opt/mamba/bin/activate cuda-base
export LD_LIBRARY_PATH=/opt/mamba/envs/cuda-base/lib:$LD_LIBRARY_PATH
```

- Use the **same pyproject.toml** (no changes needed).

No Changes to `uv sync`

The command remains `uv sync --all-extras --all-groups` in all environments.

3.5. conda-env.yaml for micromamba

```
---
name: cuda-base
channels:
  - nvidia
  - conda-forge
dependencies:
  # CUDA 12.1 (compatible with NVIDIA Driver 525+)
  - cuda-toolkit=12.1
```

- cudnn ≥ 8.9
- libcublas
- libcublas-dev
- libopenblas
- nccl
- gxx_linux-64 $\geq 12, < 14$ # For nvcc compatibility
- sysroot_linux-64 ≥ 2.17

Compatibility Notes

- **PyTorch 2.10.x** requires **CUDA 12.1** and **cuDNN 8.9**.
- **NVIDIA Driver ≥ 525** supports CUDA 12.x.
- **AMD EPYC (Rome)**: OpenBLAS outperforms MKL (included via libopenblas).

4. NVIDIA Compatibility Matrix

Component	uv-Only (GPU)	micromamba + uv (GPU)	CI (CPU)
PyTorch	2.10.**cu121 (CUDA 12.1)	2.10.**cu121 (links to system)	2.10.**cpu
CUDA Toolkit	Bundled in wheel	cuda-toolkit=12.1 (system)	N/A
cuDNN	Bundled in wheel	cudnn ≥ 8.9 (system)	N/A
Driver	≥ 525	≥ 525	N/A
.venv Size	~4 GB	~1 GB	~200 MB
Setup Command	uv sync --all-extras	micromamba env create + uv sync	uv sync --all-extras

5. Additional Notes /av

5.1. Why Not CPU-Only PyTorch Everywhere?

- **Prohibited by "Simple-as-possible" Rule**: Derived projects must use `uv sync --all-extras --all-groups unchanged` for GPU support.

5.2. Why Not Cache .venv in CI?

- **.venv is Environment-Specific**: Caching a GPU .venv (with CUDA wheels) and restoring it in CI (CPU) **fails** due to missing CUDA libraries. Even if cached separately, the **size exceeds 20 GB**.

5.3. uv vs. micromamba for CUDA

- **uv**:
 - **Pros**: Faster, simpler, no shell modifications.
 - **Cons**: CUDA wheels bloat .venv.
- **micromamba**:

- **Pros:** Smaller `.venv`, reproducible CUDA.
- **Cons:** Additional setup, slower dependency resolution.

5.4. Recommended Approach

- **For CI (int):** Use the **refactored `pyproject.toml` + `.gitlab-ci.yml`** (CPU-only wheels, no `.venv` cache).
- **For GPU (dev, test, prod):** Use **`micromamba` + `uv`** for **minimal disk usage** and **reproducibility**.
Example Dockerfile for prod:

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y micromamba
COPY conda-env.yaml .
RUN micromamba env create -n cuda-base -f conda-env.yaml
ENV PATH="/opt/mamba/bin:$PATH"
RUN uv sync --all-extras --all-groups
```

6. Footnotes

Q:2

Q:2 - TODO:(q2-title)

^

Question Prompt 2

TODO:(q1-prompt) ...

A:2 (Claude)

[^] *TODO:(a1-ref-clause)*

TODO:(a2-clause) ...

A:2 (Gemini)

[^] *TODO:(a2-ref-gemini)*

TODO:(a2-gemini) ...

A:2 (ChatGPT)

[^] *TODO:(a2-ref-chatgpt)*

TODO:(a2-chatgpt) ...

A:2 (DeepSeek)

[^] *TODO:(a2-ref-deepseek)*

TODO:(a2-deepseek) ...

A:2 (Perplexity)

[^] *TODO:(a2-ref-perplexity)*

TODO:(a2-perplexity) ...

A:2 (Vibe)

[^] *TODO:(a2-ref-vibe)*

TODO:(a2-vibe) ...

A:a

A:a - TODO:(aa-title)

^

Appendix a

TODO:(aa-text) ...

vim: set foldmethod=marker : }}} // %* -->